

Set 27: Summarizing Single Features

Skill 27.1: Identify the data type of a single feature

Skill 27.2: Select and inspect a single feature from a DataFrame

Skill 27.3: Compute measures of central tendency

Skill 27.4: Compute measures of spread

Skill 27.5: Describe and visualize the distribution of a quantitative feature

Skill 27.6: Describe and visualize a categorical feature

Skill 27.1: Identify the data type of a single feature

Skill 27.1 Concepts

Univariate summaries describe one variable (one feature, one column) at a time. This is useful for answering questions about each individual feature in a dataset.

Variables can typically be classified as *quantitative* (i.e., numeric) or *categorical* (i.e., discrete). And, depending on its type, we may want to choose different summary metrics and visuals to use.

Suppose we have the following dataset on New York City rental listings imported into a pandas DataFrame

rentals		
rent	size_sqft	borough
2550	480	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan

The DataFrame above has two quantitative variables (rent and size_sqft) and one categorical variable (borough).

The pandas library offers a handy method *describe()* for displaying some of the most common summary statistics for the columns in a DataFrame. By default, the result only includes numeric columns, but we can specify `include='all'` to the method to display categorical ones as well,

rentals			
rent	size_sqft	borough	
2550	480	Manhattan	
11500	2000	Manhattan	
3000	1000	Queens	
4500	916	Manhattan	
4795	975	Manhattan	
Code			
print(rentals.describe(include='all'))			
Output			
	rent	size_sqft	borough
count	5000.000000	5000.000000	5000
unique	NaN	NaN	3
top	NaN	NaN	Manhattan
freq	NaN	NaN	3539
mean	4536.920800	920.101400	NaN
std	2929.838953	440.150464	NaN
min	1250.000000	250.000000	NaN
25%	2750.000000	633.000000	NaN
50%	3600.000000	800.000000	NaN
75%	5200.000000	1094.000000	NaN
max	20000.000000	4800.000000	NaN

This is a great way to get an overview of all the variables in a dataset. Notice how different statistics are displayed depending on the variable type. In the rest of the lesson, we'll look more closely at the common ways to summarize and visualize quantitative and categorical variables.

[Skill 27.1 Exercise 1](#)

Skill 27.2: Select and inspect a single feature from a DataFrame

Skill 27.2 Concepts

In pandas, a feature is a column in a DataFrame. Before we summarize a feature, we need to select it and inspect it. This helps us decide *how* it should be summarized.

There are two common ways to select a single feature,

Rentals		
rent	size_sqft	borough
2550	NaN	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan
Code		
<pre>rent = rentals['rent'] size = rentals.size_sqft print(rent) print(size)</pre>		
Output		
<pre>0 2550 1 11500 2 3000 3 4500 4 4795 Name: rent, dtype: int64 0 480 1 2000 2 1000 3 916 4 975 Name: size_sqft, dtype: int64</pre>		

Both return a *Series*, which represents one column of data.

Once we have the feature selected, we can use the following checklist to inspect the feature,

- What type of data it is
- Whether values are missing
- How varied the values are

Datatype

The *dtype* attribute can be used to find the datatype,

Code
<pre>rent = rentals['rent'] size = rentals.size_sqft print(rent.dtype) print(size.dtype) print(borough.dtype)</pre>
Output
<pre>int64 int64 object</pre>

Knowing the datatype is important, because it will determine the type of summary statistics we can provide,

- Numerical data (int, float) can be summarized with *mean*, *median*, *spread*
- Non-numerical data (object, category) should be summarized with *counts* and *proportions*

Missing Values

The number of missing values can be found using the *isna()* method. Likewise, the number of non-missing values can be found using the *notna()* method.

Rentals		
rent	size_sqft	borough
2550	NaN	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan
Code		
<pre>size = rentals.size_sqft print(size.notna().sum()) print(size.isna().sum())</pre>		
Output		
<pre>4 1</pre>		

Unique Values

Understanding how many unique values a feature has helps determine whether it is *categorical* or *quantitative*. The `nunique()` method provides the number of unique values. The `unique()` method returns a list of the values.

Rentals		
rent	size_sqft	borough
2550	NaN	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan
Code		
<pre>borough = rentals.borough print(borough.unique()) print(borough.nunique())</pre>		
Output		
<pre>['Manhattan' 'Queens'] 2</pre>		

Before summarizing a feature, always inspect it to understand what kind of data it contains and whether it is clean.

This step prevents mistakes like calculating an average on categorical data or ignoring missing values.

Skill 27.2 Exercise 1

Skill 27.3: Compute measures of central tendency

Skill 27.3 Concepts

Measures of *central tendency* describe the *typical* or *center* value of a single *quantitative* feature. Common measures of center include,

- Mean
- Median
- Mode
- Trimmed Mean

Mean (average)

The *mean* is the sum of all values divided by the number of values.

```
rentals["rent"].mean()
```

The *mean* works well when data is symmetric, but is strongly affected by *outliers*.

Median (middle value)

The *median* is the middle value when data is ordered.

```
rentals["rent"].median()
```

The median is resistant to outliers and better represents “typical” values in skewed data

Mode (most common value)

The mode is the value that appears most often.

```
rentals["rent"].mode()
```

The mode is useful when values repeat. It is often used with categorical data, but works for numeric data too

Trimmed mean

A trimmed mean removes a percentage of the lowest and highest values before computing the mean. This reduces the influence of extreme values.

The example below trims the extreme 10% of values from the *rent* column

```
from scipy.stats import trim_mean  
trim_mean(rentals["rent"], 0.1)
```

The trimmed mean is useful when outliers exist but you still want an average. It is common in standardized testing and economics

The “best” measure of center depends on the shape of the data and the presence of outliers, not just convenience. Below is a summary of

The table below is a guide for selecting an appropriate measure of center,

Situation	Best Measure
Symmetric distribution	Mean
Skewed distribution	Median
Repeated values	Mode
Outliers present	Median or trimmed mean

Skill 27.3 Exercise 1

Skill 27.4: Compute measures of spread

Skill 27.4 Concepts

Measures of spread describe how spread out or variable the values of a single quantitative feature are.

Two features can have the same center but very different spread.

Range

The *range* measures the distance between the minimum and maximum values. To calculate the *range*, we can combine the *min()* and *max()* functions,

```
range_rent = rentals.rent.max() - rentals.rent.min()
```

Quartiles

Quartiles divide the data into four equal parts.

- Q1: 25% of values are below this point
- Q2: Middle value (median)
- Q3: 75% of values are below this point

Quartiles help describe how values are distributed across the dataset.

```
q1 = rentals["rent"].quantile(0.25) # Q1
q2 = rentals["rent"].quantile(0.50) # Q2 (Median)
q3 = rentals["rent"].quantile(0.75) # Q3
```

Interquartile Range (IQR)

The *IQR* measures the spread of the middle 50% of the data. To calculate the IQR we can use the quartile values from above,

```
q1 = rentals["rent"].quantile(0.25) # Q1
q3 = rentals["rent"].quantile(0.75) # Q3
IQR = q3 - q1
```

IQR tells us how spread out the central values are and is resistant to outliers. It can also be applied to identify outliers.

Standard Deviation

The *standard deviation* measures how far values typically deviate from the *mean*.

```
rent_std = rentals.rent.std()
```

Larger *standard deviation* values indicate greater variability, and smaller values indicate data clustered near the mean.

The *standard deviation* is strongly influenced by outliers, and is most useful when the data is roughly symmetric

Measure	Resistant to Outliers?	What It Best Describes
Min / Max	No	Extreme values
Range	No	Total spread
Quartiles	Yes	Distribution of values
IQR	Yes	Middle 50% spread
Standard Deviation	No	Average distance from mean

Skill 27.4 Exercise 1

Skill 27.5: Describe and visualize the distribution of a quantitative feature

Skill 27.5 Concepts

While summary statistics are helpful for quantifying a feature, they can make it difficult to understand the *overall shape* of the data. Visualizations allow us to see patterns that are hard to detect from numbers alone.

For quantitative features, *boxplots* and *histograms* are two common visualizations. These plots help us describe a distribution by showing information about center, spread, and extreme values. Histograms are especially useful for revealing patterns such as skew or multiple clusters of values.

In this section, we will use visualizations alongside summary statistics to describe the distribution of a single quantitative feature.

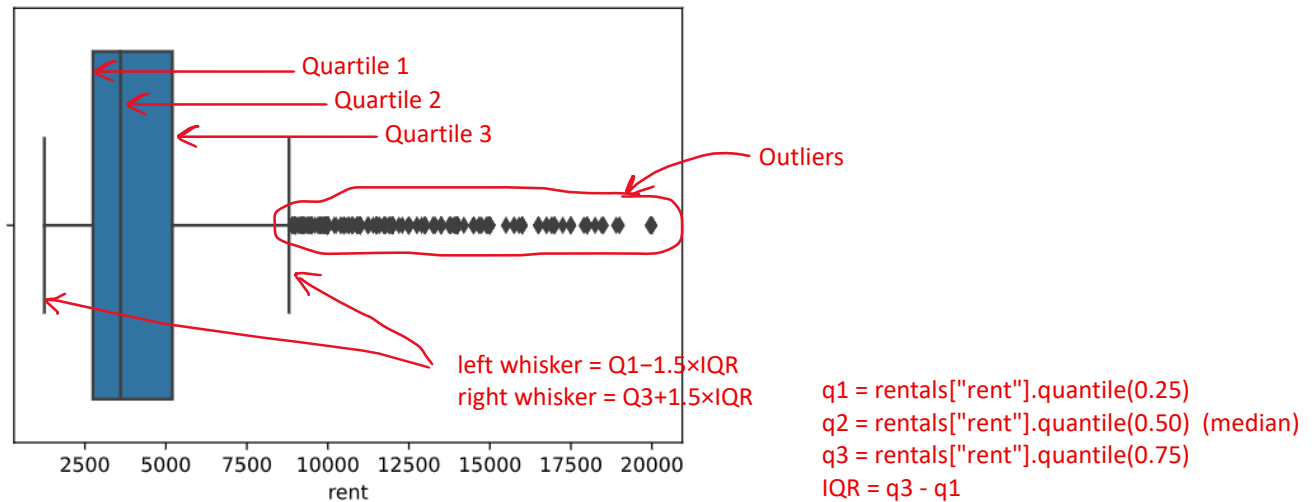
Python's seaborn library, built on top of matplotlib, provides the *boxplot()* and *histplot()* functions, which make it easy to visualize features stored in a pandas DataFrame.

Consider a box plot below which shows *rent* column in our *rentals* DataFrame.

Code

```
import matplotlib.pyplot as plt
import seaborn as sns
# Boxplot for rent
sns.boxplot(x='rent', data=rentals)
plt.show()
plt.close()
```

Output



Explanation

The boxplot shows that rent is *right-skewed*. Most rents are between ~2500 - 5000, as shown by the compact middle 50% of values within the box. However, there are several high-value outliers extending far to the right, indicating that a small number of rentals are much more expensive. Because of these outliers, the median is a better representation of the rent than the mean.

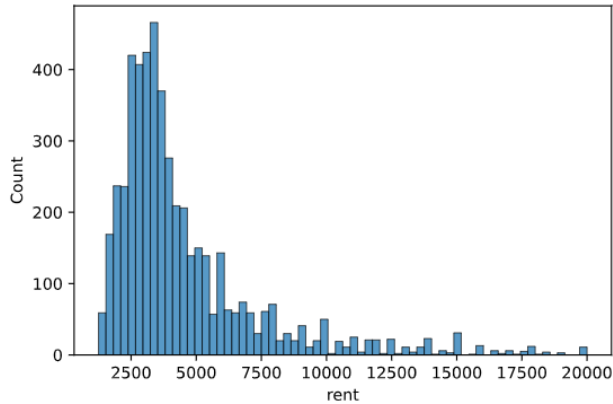
| A boxplot allows us to quickly see spread, center, and outliers at the same time.

Now, examine the histogram which shows the same column visualized in a different way,

Code

```
sns.histplot(x="rent", data=rentals, bins=10)
plt.show()
plt.close()
```

Output



Explanation

The histogram shows a right-skewed distribution. Most rentals are in the 2500 – 2800 range, which is reflected in the taller bars on the left side of the plot. A small number of rentals are much more expensive, creating a long tail to the right. This suggests that extreme values are present and that the median is a better measure of center than the mean.

Histograms help us see shape, while summary statistics help us quantify it

Skill 27.5 Exercise 1

Skill 27.6: Describe and visualize a categorical feature

Skill 27.6 Concepts

When it comes to categorical variables, the measures of central tendency and spread that work for numerical data—such as the mean or standard deviation—are generally not appropriate. Categorical values represent distinct groups, not continuous quantities, and they often do not have a meaningful numerical ordering.

Instead of asking “*What is the average?*”, we ask:

- *How often does each category occur?*
- *Which categories are most or least common?*
- *How dominant is one category compared to others?*

To answer these questions, we use counts, proportions, and bar charts.

The most common way to summarize a categorical feature is to count how often each category appears. This can be done with the `value_counts()` method.

Rentals		
rent	size_sqft	borough
2550	NaN	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan
Code		
<code>df.borough.value_counts()</code>		
Output		
Manhattan 3539 Brooklyn 1013 Queens 448		

By default, `value_counts()` returns the results sorted in descending order by count, where the top element is the *mode*, or the most frequently appearing value. In this case, the mode is Manhattan with 3,539 rental listings.

The counts returned by the `value_counts()` method tell us *how many*, but sometimes it is useful to also look at the proportion of values in each category. For example, knowing that there are 3,539 rental listings in Manhattan is hard to interpret without any context about the counts in the other categories. On the other hand, knowing that Manhattan listings make up 71% of all New York City listings tells us a lot more about the relative frequency of this category. We can calculate the proportion for each category by using the *normalize* attribute,

Rentals		
rent	size_sqft	borough
2550	NaN	Manhattan
11500	2000	Manhattan
3000	1000	Queens
4500	916	Manhattan
4795	975	Manhattan
Code		
<code>df.borough.value_counts(normalize = True)</code>		
Output		
Manhattan 0.7078 Brooklyn 0.2026 Queens 0.0896		

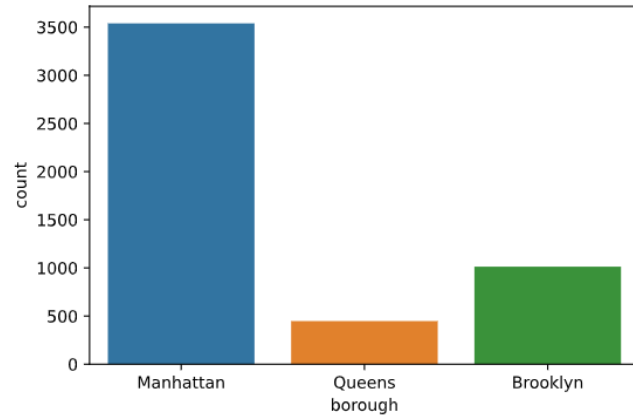
For categorical variables, bar charts and pie charts are common options for visualizing the count (or proportion) of values in each category. They can also convey the relative frequencies of each category.

Python's seaborn library offers several methods that can create bar charts. The simplest for plotting the counts is `countplot()`

Code

```
sns.countplot(x='borough', data=rentals)
plt.show()
plt.close()
```

Output



Explanation

The bar chart shows the rental counts for each borough

To describe a categorical feature, we focus on counts, proportions, and visual comparisons, not numerical averages.

Skill 27.6 Exercise 1